

Project Documentation

Workforce Management SaaS Platform

VERSION

1.0.0

DATE

May 2026

STACK

Django 5.2 + React 19

CONTENTS

- 1. Product Overview
- 2. Tech Stack
- 3. Project Structure
- 4. Roles & Permissions
- 5. Modules & Routes
- 6. API Endpoints
- 7. Database Models
- 8. GPS & Photo Rules
- 9. US FLSA Payroll
- 10. UK WTR / PAYE / NI
- 11. Compliance App
- 12. Scheduling WTR
- 13. Audit & Exports
- 14. Frontend Architecture
- 15. CSS Variables
- 16. Hard Rules

1. Product Overview

QuickTIMS is a multi-tenant, cloud-hosted workforce management SaaS platform. Organizations use it to manage employee time tracking with GPS and photo verification, shift scheduling, leave requests, payroll processing, field worker task management, and compliance reporting for both US (FLSA) and UK (WTR/PAYE) labour law regimes.

Core Pillars

- **Multi-tenancy:** Every piece of data is strictly scoped to an organization (company). No cross-tenant data leakage is permitted at any layer.
- **Dual jurisdiction:** Native support for US FLSA overtime rules (federal + California + Alaska) and UK Working Time Regulations, PAYE, NI, RTW, and RTI FPS.
- **GPS + Photo clock-in:** Employees must provide a GPS fix and a live photo before clocking in; the backend validates the GPS coordinates against the org's geofence.
- **Role separation:** Admin users see and manage all data. Employee users see only their own records.
- **Audit trail:** Every create/update/delete operation on compliance-relevant models is logged to an immutable AuditLog collection.

Platform Targets

Target	Description
SMBs 10–500 staff	Primary market — single org, multiple locations
Multi-location retail	Live GPS map, per-location geofence, shift scheduling
UK professional services	WTR opt-out management, RTI FPS, RTW document tracking
US field workforce	Task management, photo attestation, FLSA OT calculation

2. Tech Stack

Frontend

Technology	Version / Notes
React	19 — functional components, JSX only (never TSX)
Vite	Build tool and dev server
Redux Toolkit	All global/shared state — never useState for shared state
React Router v6	Always useNavigate(), never window.location
Leaflet / React-Leaflet	All maps and geolocation UI
Lucide React	All icons — no other icon library
@react-oauth/google	Google OAuth — never implement OAuth manually
Inline styles	CSS variables only — no Tailwind, no CSS modules, no UI libs

Backend

Technology	Version / Notes
Python	3.13
Django	5.2
Django REST Framework	All APIs — ViewSets or APIViews
django-mongodb-backend	MongoDB via Django ORM — no raw PyMongo
django-rest-framework-simplejwt	JWT auth — Bearer token in Authorization header
Pillow	Image handling for photo verification
reportlab	PDF generation (audit exports, DOL recordkeeping)
MongoDB	Primary database — all collections

Infrastructure Assumptions

- Deployed behind an HTTPS reverse proxy (nginx / Caddy).
- MongoDB Atlas or self-hosted replica set.
- Django MEDIA_ROOT configured for Pillow FileField photo storage.
- SMTP configured for RTW alert emails (Django EMAIL_BACKEND).
- Cron or Celery Beat for scheduled management commands (send_rtw_alerts).

3. Project Structure

The repository is organised into a top-level frontend/ and backend/ directory alongside a mnt/ folder that holds AI skill files used during development.

```
quicktims/ ■■■ CLAUDE.md                                ← AI project instructions ■■■ frontend/
■ ■■■ src/ ■ ■■■ ui/ ■ ■■■ App.jsx                      ← Route declarations
■ ■■■ routes.js                                          ← ALL route constants
■ ■■■ pages/                                             ← One file per page/route
■ ■■■ components/                                       ← Reusable UI pieces ■ ■■■ shell/
■ ■■■ AppShell.jsx                                       ← Nav + layout wrapper ■ ■■■ state/
■ ■■■ auth/                                              ← Auth slice + hooks ■ ■■■ store.js
■ ■■■ main.jsx ■■■ backend/                             ■■■ manage.py
■ ■■■ config/                                           ← Django settings
■ ■■■ accounts/                                         ← Auth, JWT, Google OAuth, roles
■ ■■■ employees/                                        ← Employee profiles
■ ■■■ time_tracking/                                    ← Clock in/out, breaks, GPS, photo
■ ■■■ leaves/                                           ← Leave requests and approvals
■ ■■■ payroll/                                          ← Payroll periods and records
■ ■■■ scheduling/                                       ← Shifts + WTR enforcement
■ ■■■ compliance/                                       ← UK WTR, FLSA, RTW, Audit, RTI
■ ■■■ reports/                                          ← Admin overview and stats
■ ■■■ tasks/                                            ← Field worker task management
```

Key Files

File	Purpose
frontend/src/ui/routes.js	Single source of truth for all route path strings
frontend/src/ui/App.jsx	React Router route declarations
frontend/src/ui/shell/AppShell.jsx	Persistent nav sidebar + layout
backend/config/settings.py	Django settings: DB, auth, email, media
backend/compliance/views.py	All compliance API views (~950 lines)
backend/scheduling/views.py	ShiftViewSet with WTR enforcement
backend/compliance/management/commands/send_rtw_alerts.py	Python RTW email command

4. Roles & Permissions

QuickTIMS uses a two-tier role system stored on the User model. All permission enforcement occurs in the Django backend — frontend role checks are purely cosmetic and must never be trusted for access control.

Role	Scope	Key Permissions
admin	Organisation-wide	Full access: all modules, all employees, all reports, settings, compliance, payroll generation, shift mana
employee	Own records only	Own timesheets, own leave requests, own tasks, own payslips, own schedule — nothing from other em

DRF Permission Classes

- **IsAuthenticated:** All endpoints require a valid JWT Bearer token.
- **IsAdminRole:** Custom permission class used on admin-only actions (create/update/delete shifts, payroll generation, compliance exports, etc.).
- **URL prefix convention:** Admin-only endpoints use /admin/ in the URL path (e.g. /api/tasks/admin/).

■ Always enforce role checks in backend serializers and views. Never rely on frontend routing to protect admin-only data.

5. Modules, Pages & Routes

All route path strings are defined in frontend/src/ui/routes.js and imported everywhere. Hardcoding path strings in pages or components is forbidden.

Route Key	Path	Page Component	Description
login	/login	LoginPage	JWT login + Google OAuth
onboarding	/onboarding	OnboardingPage	3-step org setup wizard
get_started	/get-started	GetStartedPage	Post-onboarding welcome
dashboard	/	DashboardPage	Overview, stats, quick actions
locations	/locations	LocationsPage	Office locations + geofence config
live_locations	/live-locations	LiveLocationsPage	Real-time employee GPS map
time	/time	TimePage	GPS clock in/out + photo verification + breaks
tasks	/tasks	TasksPage	Field worker task list and actions
leaves	/leaves	LeavesPage	Apply, approve, track leave requests
payroll	/payroll	PayrollPage	Payroll periods, records, payslips
scheduling	/scheduling	SchedulingPage	Employee shift scheduling
employees	/employees	EmployeesPage	Employee directory, profiles, roles
reports	/reports	ReportsPage	Admin overview and analytics
compliance	/compliance	CompliancePage	UK/US compliance dashboard (admin)
settings	/settings	SettingsPage	Org settings, roles, configuration

Settings Sub-routes

The settings area has deep sub-routes, all defined in routes.js and rendered via the SettingsPage component with a section prop, or via dedicated sub-page components:

Route Key	Path	Component
settings_people	/settings/people	PeopleSettingsPage
settings_timetracking	/settings/time-tracking	TimeTrackingSettingsPage
settings_attendance	/settings/attendance	SettingsPage (section=attendance)
settings_schedules	/settings/schedules	WorkSchedulesSettingsPage
settings_holidays	/settings/holidays	HolidaysSettingsPage
settings_location	/settings/location	LocationsSettingsPage
settings_payroll	/settings/payroll	SettingsPage (section=payroll)
settings_notifications	/settings/notifications	SettingsPage (section=notifications)
settings_security	/settings/security	SettingsPage (section=security)
settings_rbac	/settings/rbac	SettingsPage (section=rbac)
settings_billing	/settings/billing	SettingsPage (section=billing)

6. API Endpoints

Auth — /api/auth/

Method	Endpoint	Description
POST	/api/auth/login/	JWT login — returns access + refresh tokens
POST	/api/auth/register/	User registration
POST	/api/auth/google/	Google OAuth login/registration
POST	/api/auth/refresh/	JWT token refresh
GET	/api/auth/me/	Current user profile (requires Bearer token)

Employees — /api/employees/

Method	Endpoint	Description
GET	/api/employees/	List all employees (admin sees all, employee sees self)
POST	/api/employees/	Create employee (admin only)
GET	/api/employees/<id>/	Employee detail
PUT	/api/employees/<id>/	Full update
PATCH	/api/employees/<id>/	Partial update
DELETE	/api/employees/<id>/	Delete employee

Time Tracking — /api/time/

Method	Endpoint	Description
POST	/api/time/clock-in/	Clock in with GPS coordinates + photo
POST	/api/time/clock-out/	Clock out
POST	/api/time/break/start/	Start break (must be clocked in)
POST	/api/time/break/end/	End break
GET	/api/time/timesheets/	View timesheet summary
GET	/api/time/logs/	List time logs (own only for employees)
POST	/api/time/logs/	Create time log
GET	/api/time/logs/<id>/	Time log detail
PUT	/api/time/logs/<id>/	Edit time log
PATCH	/api/time/logs/<id>/	Partial edit
DELETE	/api/time/logs/<id>/	Delete time log
GET	/api/time/admin/employees/<eid>/logs/	Admin: all logs for employee

Leaves — /api/leaves/

Method	Endpoint	Description
GET	/api/leaves/	List leave requests

Method	Endpoint	Description
POST	/api/leaves/	Create leave request
GET	/api/leaves/<id>/	Leave detail
PUT	/api/leaves/<id>/	Update leave request
PATCH	/api/leaves/<id>/	Approve or reject
DELETE	/api/leaves/<id>/	Delete leave request

6. API Endpoints (continued)

Payroll — /api/payroll/

Method	Endpoint	Description
POST	/api/payroll/generate/	Generate payroll for a period (admin only)
GET	/api/payroll/records/	List payroll records
POST	/api/payroll/records/	Create record
GET	/api/payroll/records/<id>/	Record detail
PUT	/api/payroll/records/<id>/	Update record
PATCH	/api/payroll/records/<id>/	Partial update
DELETE	/api/payroll/records/<id>/	Delete record

Scheduling — /api/scheduling/

Method	Endpoint	Description
GET	/api/scheduling/shifts/	List shifts (admin: all, employee: own)
POST	/api/scheduling/shifts/	Create shift — WTR checked for UK employees
GET	/api/scheduling/shifts/<id>/	Shift detail
PUT	/api/scheduling/shifts/<id>/	Update shift — WTR re-checked
PATCH	/api/scheduling/shifts/<id>/	Partial update — WTR re-checked
DELETE	/api/scheduling/shifts/<id>/	Delete shift

Tasks — /api/tasks/

Method	Endpoint	Description
GET	/api/tasks/admin/	Admin: list all tasks
POST	/api/tasks/admin/	Admin: create task
GET	/api/tasks/admin/<pk>/	Admin: task detail
PUT	/api/tasks/admin/<pk>/	Admin: update task
PATCH	/api/tasks/admin/<pk>/	Admin: partial update
DELETE	/api/tasks/admin/<pk>/	Admin: delete task
POST	/api/tasks/admin/<pk>/attachments/	Admin: add attachment to task
GET	/api/tasks/my/	Employee: my assigned tasks
POST	/api/tasks/my/<pk>/<action>/	Employee: start or complete task

Compliance — /api/compliance/

Method	Endpoint	Description
GET	/api/compliance/ot-risk/	OT risk dashboard — employees near OT threshold
GET	/api/compliance/uk-48hr/	UK 48-hour monitor — all employees

Method	Endpoint	Description
GET	/api/compliance/uk-48hr/<eid>/	UK 48-hour detail for one employee
GET	/api/compliance/holiday-accrual/	List holiday accrual records
POST	/api/compliance/holiday-accrual/accrue/<eid>/	Accrue holiday for employee
GET	/api/compliance/rtw/	List RTW documents
POST	/api/compliance/rtw/expiry-check/	Check for expiring RTW docs
POST	/api/compliance/rtw/send-alerts/	Send RTW expiry alert emails
GET	/api/compliance/wtr-optout/	List WTR opt-out agreements
POST	/api/compliance/wtr-optout/	Create WTR opt-out agreement
GET	/api/compliance/audit-log/	List audit log entries
GET	/api/compliance/audit-log/export/	Export audit log as PDF
POST	/api/compliance/break-attestation/	Submit break attestation
GET	/api/compliance/wage-floor/	Check NMW/minimum wage compliance
GET	/api/compliance/exempt-status/<eid>/	FLSA exempt status for employee
GET	/api/compliance/break-compliance/	Break compliance report
GET	/api/compliance/rti-fps/	HMRC RTI Full Payment Submission export

Reports — /api/reports/

Method	Endpoint	Description
GET	/api/reports/overview/	Admin overview statistics (headcount, hours, OT, leaves)

7. Database Models (MongoDB Collections)

All models use Django ORM with the django-mongodb-backend. Raw PyMongo is never used. Every queryset must be scoped to the requesting org (company) or user.

App	Model	Key Fields	Purpose
accounts	User	email, role, company, is_active	Auth credentials, role, org membership
accounts	Company	name, plan, geofence_radius	Organisation/tenant record
employees	Employee	user, country, hourly_rate, hire_date	Employee profile and pay rate
time_tracking	TimeLog	employee, clock_in, clock_out, latitude, longitude, photo	Clock in/out session with GPS + photo
time_tracking	Break	timelog, start, end	Break within a TimeLog session
leaves	LeaveRequest	employee, leave_type, start_date, end_date, status	Leave application and approval
payroll	PayrollPeriod	company, start_date, end_date	Defines a pay period
payroll	PayrollRecord	employee, period, gross_pay, net_pay, tax, ni	Calculated pay for one employee per period
scheduling	Shift	employee, shift_start, shift_end, location	Scheduled shift
tasks	Task	employee, title, status, due_date, location	Field worker job assignment
tasks	TaskAttachment	task, file, uploaded_at	File or photo for a task
compliance	WTROptOut	employee, signed_date, is_active	UK WTR 48hr opt-out agreement
compliance	RightToWork	employee, doc_type, expiry_date, alert_sent_*	UK RTW document tracking
compliance	HolidayAccrual	employee, accrued_days, used_days	UK statutory holiday balance
compliance	AuditLog	action, model_name, object_id, changed_by, timestamp, snapshot	Immutable audit record

AuditLog Immutability

The AuditLog model overrides save() to raise ValueError if a pk already exists, preventing any modification after creation. A pre_delete signal captures a full snapshot of the record being deleted before the AuditLog entry is written.

Multi-tenancy Enforcement

■ Every queryset must filter by company (org) and/or user. Unscoped queries are a critical security violation. The backend never trusts a company_id from request body — it always reads it from request.company set by middleware.

8. GPS & Photo Clock-In Rules

Frontend Rules (TimePage.jsx)

- Use **Leaflet / React-Leaflet** for live map preview during clock-in.
- Use **browser navigator.geolocation** — always with **enableHighAccuracy: true, timeout: 10000**.
- Show GPS status: **Acquiring** → **Locked** → **Error**.
- Capture photo via device camera (getUserMedia or file input with capture=camera).
- **Disable clock-in button** until GPS is locked AND photo is captured.
- Send { **latitude, longitude, accuracy, photo** } as multipart form data on clock-in.

Backend Rules (time_tracking app)

- Validate GPS coordinates against org geofence on every clock-in.
- Reject clock-in if outside geofence radius — return **HTTP 403**.
- Store photo using **Pillow + Django FileField** — never base64 in MongoDB.
- Prevent duplicate clock-in: employee must clock out before clocking in again — return **HTTP 400**.
- Record break start/end only within an active (not yet clocked-out) TimeLog session.

Geofence Calculation

The backend calculates the Haversine distance between the employee's submitted GPS coordinates and each of the org's configured location points. If the minimum distance exceeds the org's geofence_radius (in metres), the clock-in is rejected.

■ GPS coordinates from the client are not trusted for anything other than geofence validation. The canonical location is always stored as received — accuracy field is logged for audit purposes.

9. US FLSA Payroll Engine

The payroll engine implements US Federal FLSA overtime, California daily overtime, and Alaska daily overtime rules. All calculations run server-side in the payroll app.

Federal FLSA Rules

Hours Band	Rate	Rule
0–40 hours/week	1.0× regular rate	Straight time
>40 hours/week	1.5× regular rate	FLSA weekly overtime (29 USC §207)

California Daily OT (Lab. Code §510)

Hours Band	Rate	Rule
0–8 hours/day	1.0×	Straight time
8–12 hours/day	1.5×	CA daily overtime
>12 hours/day	2.0×	CA double time
7th consecutive day, first 8h	1.5×	7th day premium
7th consecutive day, >8h	2.0×	7th day double time

Alaska Daily OT (AS 23.10.060)

Hours Band	Rate
0–8 hours/day	1.0×
>8 hours/day	1.5×

FLSA Exempt Status

An employee is FLSA-exempt if they meet BOTH conditions: (1) salaried basis, AND (2) salary $\geq$ \$844/week (as of 2024 rule, 29 CFR Part 541). The `/compliance/exempt-status/` endpoint checks these conditions and returns a detailed breakdown including the applicable threshold.

Payroll Generation Flow

- Admin selects a PayrollPeriod (start/end dates).
- Backend fetches all TimeLogs for all active employees within the period.
- For each employee, determine jurisdiction (US-state or UK).
- Apply appropriate OT rules to compute gross pay.
- For US: no tax deduction at this layer (gross pay only, tax handled externally).
- For UK: apply PAYE income tax + NI deductions to arrive at net pay.
- Create or update PayrollRecord for each employee.
- Return summary payload with totals.

10. UK WTR / PAYE / NI Engine

Working Time Regulations 1998

Regulation	Rule	Enforcement Point
Reg 4 — 48hr average	17-week rolling average of weekly hours must not exceed 48	ShiftViewSet.perform_create / perform_update
Reg 4 — opt-out	Employee may sign WTR opt-out to exceed 48h average	WTROptOut model; _check_wtr_opt_out() helper
Reg 10 — daily rest	Minimum 11 hours rest between consecutive shifts	ShiftViewSet — always enforced even with opt-out
Reg 13 — annual leave	28 days statutory leave p.a. (20 days + 8 bank holidays)	HolidayAccrual model; 12.07% accrual rate
Reg 13A — additional	Additional contractual leave beyond 28 days minimum	HolidayAccrual.additional_days field

17-Week Rolling Average Calculation

When creating or updating a shift for a UK employee without an active opt-out, the system computes a 17-week rolling average as follows:

- For each of the 17 weeks ending today: fetch actual clocked hours AND scheduled hours.
- Take **max(actual, scheduled)** per week to avoid double-counting.
- Add the proposed new shift's hours to the week it falls in.
- Compute average = sum(17 weeks) / 17.
- If average > 48.0, raise ValidationError citing WTR 1998 Reg 4.

UK PAYE Income Tax Bands (2024-25)

Band	Annual Income	Rate
Personal Allowance	£0 – £12,570	0%
Basic Rate	£12,571 – £50,270	20%
Higher Rate	£50,271 – £125,140	40%
Additional Rate	Over £125,140	45%

UK National Insurance (Class 1 — Category A)

Threshold	Annual	Employee Rate	Employer Rate
Below Lower Earnings Limit	< £6,396	0%	0%
LEL – Primary Threshold	£6,396 – £12,570	0% (credit only)	0%
PT – Upper Earnings Limit	£12,570 – £50,270	8%	13.8%
Above UEL	> £50,270	2%	13.8%

National Minimum Wage (April 2024)

Age Group	NMW Rate
21 and over (National Living Wage)	£11.44 / hour
18–20	£8.60 / hour
16–17	£6.40 / hour
Apprentice	£6.40 / hour

UK PAYE Annualisation

Period gross pay is annualised ($\times 52 / \text{weeks_in_period}$) before applying tax and NI bands. The resulting period tax and NI are de-annualised back to the pay period. This ensures correct band application regardless of period length.

11. Compliance Application

The compliance Django app consolidates all regulatory monitoring and export functionality. It is admin-only — all views check `IsAdminRole`.

Views Overview

View Class	Endpoint	Purpose
<code>OTRiskDashboardView</code>	<code>/compliance/ot-risk/</code>	Employees approaching or exceeding OT threshold this week
<code>UK48HrMonitorView</code>	<code>/compliance/uk-48hr/</code>	17-week rolling average for all UK employees
<code>HolidayAccrualViewSet</code>	<code>/compliance/holiday-accrual/</code>	CRUD for holiday accrual records
<code>EmployeeHolidayAccrualView</code>	<code>/compliance/holiday-accrual/accrue/<id>/</code>	Run accrual calculation for one employee
<code>RightToWorkViewSet</code>	<code>/compliance/rtw/</code>	CRUD for RTW documents
<code>RTWExpiryCheckView</code>	<code>/compliance/rtw/expiry-check/</code>	List documents expiring within 60 days
<code>RTWAlertEmailView</code>	<code>/compliance/rtw/send-alerts/</code>	Send email alerts for expiring RTW docs
<code>WTROptOutViewSet</code>	<code>/compliance/wtr-optout/</code>	CRUD for WTR 48hr opt-out agreements
<code>AuditLogListView</code>	<code>/compliance/audit-log/</code>	Paginated audit log (admin only)
<code>AuditLogExportView</code>	<code>/compliance/audit-log/export/</code>	Export audit log as reportlab PDF
<code>BreakAttestationView</code>	<code>/compliance/break-attestation/</code>	Submit or retrieve break attestation
<code>WageFloorCheckView</code>	<code>/compliance/wage-floor/</code>	Compare employee pay to NMW/NLW
<code>ExemptStatusView</code>	<code>/compliance/exempt-status/<id>/</code>	FLSA exempt status for one employee
<code>BreakComplianceReportView</code>	<code>/compliance/break-compliance/</code>	Report on break violations
<code>AuditLogExportView</code>	<code>/compliance/audit-log/export/</code>	PDF audit log export (DOL 3-year retention)
<code>RTIFPSExportView</code>	<code>/compliance/rti-fps/</code>	HMRC RTI Full Payment Submission JSON
<code>RTWAlertEmailView</code>	<code>/compliance/rtw/send-alerts/</code>	RTW expiry email alerts at 60/30/7d

RTW Expiry Alert System

Right to Work documents are tracked per employee. The system fires alerts at three thresholds before expiry: 60 days, 30 days, and 7 days. Each threshold has an independent boolean flag (`alert_sent_60d` / `alert_sent_30d` / `alert_sent_7d`) on the `RightToWork` model to ensure each alert is sent exactly once. The `send_rtw_alerts` management command is designed to be run by cron or Celery Beat — it is idempotent and supports a `--dry-run` flag.

DOL Audit Log PDF Export

The `AuditLogExportView` generates a reportlab PDF of all audit log entries for the org. The PDF includes a legal attestation header citing DOL 29 CFR Part 516 (3-year minimum recordkeeping), a timestamp, and a paginated table of all audit entries. The Content-Disposition header is set to attachment for browser download.

12. Scheduling & WTR Enforcement

The scheduling app manages employee shifts. ShiftViewSet enforces UK Working Time Regulations on every create and update for UK employees.

WTR Check Flow

For each create or update on a Shift:

- 1. Check employee.country — if not 'UK', skip all WTR checks entirely.
- 2. **11-hour rest (Reg 10)**: always enforced regardless of opt-out. Look for existing shifts within a ±35-hour window. If any shift leaves < 11h gap before or after the new shift, raise ValidationError.
- 3. **48-hour average (Reg 4)**: check _check_wtr_opt_out(). If employee has an active WTR opt-out, skip. Otherwise, compute 17-week rolling average including the new shift and raise ValidationError if average > 48h.

Key Helper Functions

Function	Purpose
_shift_hours(start, end)	Decimal hours from timedelta — accurate to 4dp
_get_scheduled_weekly_hours(emp, y, w)	Total scheduled hours for an ISO week
_get_actual_weekly_hours(emp, y, w)	Total clocked hours for an ISO week from TimeLogs
_build_17_week_hours_with_new_shift(...)	17-element list of weekly hours including proposed shift
_check_wtr_opt_out(employee)	True if employee has active WTROptOut record
_check_11h_rest(emp, start, end)	Returns (is_ok, conflicting_shift) — checks ±35h window

Error Messages

ValidationError messages include the regulation citation so the admin has full context:

```
{"shift_start": "UK WTR violation: Less than 11 hours rest between this shift and existing shift 2026-05-08 22:00 - 06:00. Minimum 11 hours rest required (WTR 1998 Reg 10)."}

{"shift_start": "UK WTR violation: This shift would push the 17-week rolling average to 49.2 hrs/week, exceeding the 48-hour limit. The employee must sign a WTR opt-out agreement first. (WTR 1998 Reg 4)"}
}
```

Employee Queryset Scoping

get_queryset() scopes to request.company at all times. Admins see all shifts for the org. Employees see only their own shifts. The retrieve() action adds an additional check that the authenticated employee can only view their own shift detail.

13. Audit Trail & Regulatory Exports

Audit Log Design

- Every compliance-relevant create/update/delete operation writes an AuditLog entry.
- AuditLog.save() raises ValueError if pk already exists — records are **append-only**.
- Each entry captures: action (create/update/delete), model_name, object_id, changed_by (user), timestamp (UTC), and a full JSON snapshot of the record at time of event.
- Pre-delete signals capture the full snapshot before deletion so the AuditLog entry reflects the record's state at time of deletion.

DOL PDF Export (AuditLogExportView)

GET /compliance/audit-log/export/ streams a PDF via reportlab Platypus. The document includes:

- Organisation name, export timestamp, and exporting user in the header.
- Legal attestation statement citing **DOL 29 CFR Part 516** (3-year recordkeeping requirement) and **UK WTR 1998**.
- Paginated table with columns: Timestamp | Action | Model | Object ID | Changed By.
- Response header: Content-Disposition: attachment; filename=audit_log_YYYYMMDD.pdf

HMRC RTI Full Payment Submission (RTIFPSExportView)

GET /compliance/rfi-fps/ returns a JSON payload structured for HMRC RTI FPS submission (2024-25 schema). The payload includes:

- **fps_schema_version**: '2024-25'
- **employer**: PAYE reference, Accounts Office reference, org name
- **submission**: type='FPS', tax_year, generation timestamp, submission_reason
- **employees[]**: NI number, name, payroll ID, payment date, period gross/tax/NI, YTD totals, NI category
- **totals**: aggregate gross/tax/employee_ni/employer_ni across all included employees

RTW Alert Email Command

```
python manage.py send_rtw_alerts [--dry-run] [--days N]
# Checks all RightToWork documents for the organisation # Sends alerts at 60d, 30d, 7d before expiry
# Each threshold sent exactly once (idempotent flags) # Emails all admin users for the company
# --dry-run: print alerts without sending emails
# Legal citation: Immigration, Asylum and Nationality Act 2006
```

14. Frontend Architecture

Application Shell

AppShell.jsx is the persistent layout wrapper. It renders the sidebar navigation and an for the active page. The sidebar is role-aware — admin-only items (Compliance, Reports, Live Locations) are hidden for employee users.

State Management

State Type	Solution	Rule
Auth (user, token, isReady)	Redux Toolkit slice	Persisted to localStorage with quicktims. prefix
Shared app data	Redux Toolkit slices	All shared/global state — never useState
Component-local UI state	useState	Only for UI that does not need to be shared
Form state	Local useState or Redux depending on complexity	Complexity-driven form state

Routing

React Router v6 is used exclusively. Routes are declared in App.jsx. All path strings are imported from routes.js — hardcoded strings are forbidden. Navigation uses useNavigate() — window.location is forbidden.

CompliancePage.jsx Structure

The compliance page (903 lines) is admin-only and organises all compliance views behind a five-tab navigation:

Tab	Panels
Overview	OTRISKPanel, UK48HrPanel, WageFloorPanel, BreakCompliancePanel
Right to Work	RTWPanel, WTROptOutPanel
Holiday Accrual	HolidayAccrualPanel
Audit Trail	AuditTrailPanel (paginated, 30/page, PDF download)
RTI / FPS	RTIFPSPanel (generate payload, summary cards, JSON download)

Animation Convention

Every page entry must include a fadeUp animation:

```
style={{ animation: "fadeUp 0.4s ease both" }}
```

Button & Form Class Conventions

Element	className
Primary button	btn btnPrimary
Ghost/secondary button	btn btnGhost
Form field wrapper	field
Form label	fieldLabel
Input element	input

15. CSS Variables & Design System

All colours in the application are expressed via CSS custom properties. Hardcoded hex values are only permitted in CLAUDE.md documentation — never in JSX inline styles or component code.

Semantic Variables

Variable	Meaning	Light Mode Value
var(--bg)	Page background	#f8fafc
var(--surface)	Card / panel background	#ffffff
var(--fg)	Primary text	#0f172a
var(--muted)	Secondary / hint text	#94a3b8
var(--stroke)	Border colour	#e2e8f0
var(--stroke2)	Subtle border / divider	#f1f5f9

Brand Accent Colours

Purpose	Hex	Usage
Primary (Indigo)	#5d5fef	Buttons, active nav, links, chart lines
Success (Green)	#059669	Clocked in, approved status, complete task
Danger (Red)	#E94560	Errors, clocked out, rejected, compliance alerts
Warning (Amber)	#f59e0b	Pending status, in-review, OT risk
Dark background	#0e1116	Dark mode page bg, cover page
Dark mid	#1e1b4b	Dark mode card bg, secondary accent

Icon Library

All icons must use **Lucide React**. No other icon library is permitted. Import icons individually: from 'lucide-react'. Compliance-specific icons: ShieldAlert (compliance nav item, colour #E94560).

Component Philosophy

- Build every component from scratch — no external UI libraries (no shadcn, no MUI, no Chakra).
- Inline styles only — no Tailwind utility classes, no CSS modules, no global class names.
- CSS variables for all colour and surface references.
- Components must be responsive without a CSS framework — use flexbox and CSS grid via inline styles.

16. Hard Rules — Never Violate

These rules are absolute constraints. Any code that violates them must be refactored before merging, regardless of whether it 'works'.

Backend Hard Rules

Rule	Rationale
No raw PyMongo — always Django ORM with mongodb backend	Consistency, query safety, migration support
No unscoped queries — always filter by org/user	Multi-tenant data isolation is a security requirement
No plain text passwords — use Django built-in hashing	Security — never store or compare raw passwords
No base64 images in MongoDB — use Pillow + FileField	Performance and storage efficiency
No Pydantic models — use DRF Serializers for all validation	Stack consistency — one validation layer
All datetimes UTC, stored as ISO 8601	Timezone correctness across jurisdictions
JWT only — never session auth	Stateless API for React SPA

Frontend Hard Rules

Rule	Rationale
Functional components only — no class components	React 19 best practice
Redux Toolkit for all shared state — no prop drilling	Predictable state management
Import all routes from routes.js — no hardcoded strings	Single source of truth for routing
useNavigate() only — never window.location	SPA navigation without full reload
Inline styles + CSS variables only — no Tailwind/CSS modules	Zero build-time CSS dependencies
No external UI libraries — build from scratch	Full design control, no dependency drift
JSX only — never TSX	Project is JavaScript, not TypeScript

Business Logic Hard Rules

Rule	Rationale
No GPS clock-in without photo verification	Fraud prevention and attendance integrity
No duplicate clock-in without clock-out	Data integrity — prevents overlapping sessions
No employee accessing another employee's data	Privacy and role separation
WTR 11h rest enforced even with opt-out (UK)	Reg 10 minimum rest is not opt-outable
AuditLog entries are append-only (immutable)	Regulatory requirement — audit trail integrity
RTW alert flags are one-way (never reset to False)	Alert idempotency — never re-spam admin email

■ These rules reflect legal, security, and architectural decisions made at the platform level. Do not override them for feature convenience. If a rule needs revisiting, document the discussion in CLAUDE.md before making any change.